

Okay, let's dive deep into the **Plan** entity. The goal is a simple yet comprehensive design that's realistically implementable and covers all your scenarios.

The core idea is that a **Plan** is a **conceptual bucket for money, defined by its purpose (type) and whose financial state over time is primarily understood through projections.**

Сущность **Plan** (План)

Поля (Fields):

- **id**: Уникальный идентификатор (Primary Key).
 - **user_id**: Связь с пользователем (Foreign Key to **User**).
 - **name**: **String** – Имя плана (например, "Отпуск Бали 2026", "Продукты на Июнь", "Резервный фонд").
 - **type**: **Enum (String)** – Ключевое поле, определяющее поведение и назначение плана:
 - **goal**: **Цель накопления** – Накопление на конкретную цель с целевой суммой и/или датой (например, "Новый телефон"). Ожидается накопление и затем, возможно, полное расходование.
 - **fund**: **Фонд** – Постоянное накопление или поддержание средств для общей цели, часто без фиксированной конечной даты или суммы (например, "Резервный фонд", "Инвестиционный портфель").
 - **budget_category**: **Бюджетная категория** – Представляет область расходов на определенный период (например, "Продукты на месяц", "Развлечения"). Обычно периодически пополняется и расходуется.
 - **planned_major_expense**: **Планируемый крупный расход** – Конкретный, как правило, разовый крупный расход, на который идет накопление, а затем полное расходование средств (например, "Первоначальный взнос по ипотеке"). Похож на **goal**, но с четким событием траты.
 - **target_amount**: **Decimal** (nullable) – Целевая сумма, в основном для **type = 'goal'** или **type = 'planned_major_expense'**.
 - **target_date**: **Date** (nullable) – Целевая дата, в основном для **type = 'goal'** или **type = 'planned_major_expense'**.
 - **parent_plan_id**: **Integer** (nullable, Foreign Key to **Plan.id**) – Для построения иерархии планов.
 - **status**: **Enum (String)** – Текущий статус плана ('active' – активен, 'achieved' – достигнут, 'on_hold' – на паузе, 'archived' – архивирован, 'cancelled' – отменен).
 - **priority**: **Integer** (default: 0) – Приоритет плана. Используется **DistributionRule** для определения очередности пополнения планов при ограниченных ресурсах. Более высокое значение = более высокий приоритет.
 - **notes**: **Text** (nullable) – Дополнительные заметки пользователя по плану.
 - **created_at**: **Timestamp** – Дата и время создания.
 - **updated_at**: **Timestamp** – Дата и время последнего обновления.
-

Балансы Сущности **Plan**:

У плана нет одного простого поля "баланс". Его финансовое состояние — это динамическая картина, особенно в будущем.

1. **ProjectedBalance(date, scenario_tag) (Прогнозируемый Баланс на Дату по Сценарию):**

- **Это не хранимое поле, а функция/расчет.** Возвращает ожидаемый виртуальный баланс плана на указанную **date** в рамках определенного **scenario_tag** (например, 'baseline', 'optimistic').
- **Смысл:** Это основной способ видеть состояние плана во времени, особенно для будущих периодов. Именно эти данные используются для построения графиков (Вьюхи 3, 8, 9 и т.д.).
- **Расчет:**
 1. Определяется начальная точка. Для простоты, можно считать, что для прогнозов, начинающихся "с сегодня", начальный виртуальный баланс плана равен его **CurrentActualAllocatedAmount** (см. ниже).
 2. Далее, с этой начальной точки до запрашиваемой **date**, хронологически применяются все релевантные финансовые события для данного плана и сценария:
 - **Входящие потоки:**
 - Срабатывание **DistributionRule** (правил распределения), направляющих средства в этот **Plan** (из общего пула прогнозируемых доходов или из других планов через трансферы).
 - **Исходящие потоки:**
 - Срабатывание **ForecastEntry** (прогнозных записей) типа 'expense', которые напрямую связаны с этим **Plan** (**ForecastEntry.linked_plan_id**).
 - Срабатывание **DistributionRule** типа 'plan_to_plan_transfer', переводящих средства из этого **Plan** в другие.
- Этот расчет позволяет видеть, как будет меняться баланс плана при заданных условиях.

2. **CurrentActualAllocatedAmount (Текущая Фактически Выделенная Сумма):**

- **Это также расчетное значение (на "сегодня").** Показывает, сколько *реальных, уже проведенных* денег на текущий момент концептуально выделено на данный план. Это не про будущее, а про текущее состояние на основе прошлых фактических операций.
- **Смысл:** Важно для Вьюхи 1 ("Планируемые расходы... на которые есть фактические средства") и Вьюхи 2 ("Распределение фактических денег по планам"). Помогает понять, какая часть общего фактического богатства пользователя "сидит" в этом конкретном плане.
- **Расчет:** Сумма всех фактических пополнений этого плана минус сумма всех фактических трат из него. Это определяется через:
 - **TransactionPlanLink:** Сумма **amount_linked** для всех входящих фактических транзакций минус сумма для исходящих.
 - Или через записи **DistributionRule**, которые сработали на основе фактических **Transaction** (если правила могут оперировать и фактическими деньгами для распределения).
- Для производительности это значение может кэшироваться, но логически оно является результатом агрегации фактических связей.

Правила Миграции Денег (Как Деньги Попадают в План и Уходят из Него):

Движение денег может быть **виртуальным/прогнозным** (для будущего, на основе **ForecastEntry** и **DistributionRule**) или **фактическим** (на основе **Transaction** и их связи с планами).

1. Прогнозное Поступление Средств в План (Виртуальное):

- **Через `DistributionRule` (основной способ):**
 - Правило типа `income_distribution_to_plan` направляет часть общего прогнозируемого дохода (из `ForecastEntry` типа 'income') в данный `Plan`.
 - Правило типа `plan_to_plan_transfer` переводит прогнозируемые средства из другого `Plan` (указанного как `source_plan_id`) в данный `Plan` (`destination_plan_id`) на определенную будущую дату или по расписанию.

2. Прогнозный Расход Средств из Плана (Виртуальный):

- **Через `ForecastEntry` напрямую:** Прогнозная запись `ForecastEntry` типа 'expense' с полем `linked_plan_id`, указывающим на данный `Plan` (например, "Покупка авиабилетов" для плана "Отпуск Бали"). Это напрямую уменьшает прогнозируемый баланс плана.
- **Через `DistributionRule` (трансфер из плана):** Правило типа `plan_to_plan_transfer` переводит прогнозируемые средства из данного `Plan` (он будет `source_plan_id`) в другой `Plan`.

3. Фактическое Поступление Средств в План:

- **Через `TransactionPlanLink`:** Фактическая доходная `Transaction` связывается с данным `Plan` через запись `TransactionPlanLink`. Это означает, что реальные деньги теперь числятся за этим планом. `CurrentActualAllocatedAmount` плана увеличивается.
- **Через `DistributionRule` для фактических средств:** Если система поддерживает применение `DistributionRule` к фактическим `Transaction` (например, "10% от каждой фактической зарплаты автоматически на план X"), то приход фактической зарплаты вызовет такое распределение, увеличивая `CurrentActualAllocatedAmount` плана X.

4. Фактический Расход Средств из Плана:

- **Через `TransactionPlanLink`:** Фактическая расходная `Transaction` связывается с данным `Plan`. Это показывает, что план "оплатил" этот расход. `CurrentActualAllocatedAmount` плана уменьшается.

Поведение Плана в Зависимости от Типа (`Plan.type`):

- **`goal` (Цель накопления) / `planned_major_expense` (Планируемый крупный расход):**
 - **Поступления:** Обычно пополняются через `DistributionRule` ("Откладывать X в месяц на Y").
 - **Поведение баланса:** Ожидается рост до `target_amount`. После достижения (или наступления `target_date` для `planned_major_expense`), `ForecastEntry` (расход), связанный с этим планом, обычно обнуляет его баланс. Для `goal` пользователь может затем его архивировать.
 - **Проверка для Вьюхи 1:** `planned_major_expense` отображается, если его `CurrentActualAllocatedAmount` > 0 ИЛИ если общая сумма средств пользователя (фактическая) достаточна для покрытия связанного с ним будущего `ForecastEntry` расхода.
 - **Ключевые метрики:** Прогресс к `target_amount` (%), прогнозируемая дата достижения.

- **fund (Фонд):**

- **Поступления:** Пополняется через **DistributionRule**, часто на регулярной основе.
- **Поведение баланса:** Обычно ожидается рост или поддержание определенного уровня. Может не иметь **target_amount**. Расходы из фонда – это, как правило, трансферы на другие планы или покрытие крупных непредвиденных фактических расходов (связанных через **TransactionPlanLink**).
- **Ключевые метрики:** Текущий прогнозируемый баланс, темпы роста.

- **budget_category (Бюджетная категория):**

- **Поступления:** Обычно периодически пополняется через **DistributionRule** (например, "5000 на 'Продукты' 1-го числа каждого месяца").
- **Поведение баланса:** Ожидается, что средства будут потрачены в течение периода. Неуточненные расходы (например, "средние траты на еду в неделю") могут моделироваться через **ForecastEntry** типа 'expense', связанный с этим планом, для уменьшения его прогнозного баланса. Фактические **Transaction** связываются через **TransactionPlanLink**.
- **Ключевые метрики:** Остаток бюджета на период, сравнение фактических трат с планом.
- Пример "карманные, неуточненно": План "Карманные расходы" (**type='budget_category'**). Регулярный **ForecastEntry** типа 'expense' (например, "-1000 в неделю") уменьшает его прогнозный баланс. Фактическое снятие наличных или мелкие траты линкуются к этому плану.

Дочерне-Родительские Отношения (**parent_plan_id**):

1. Агрегация Балансов:

- **ProjectedBalance(date, scenario_tag)** родительского плана может отображаться как сумма его собственного "выделенного" баланса (если он используется не только как папка) ПЛЮС сумма **ProjectedBalance** всех его активных дочерних планов.
- Это позволяет создавать иерархические отчеты (например, план "Все накопления" показывает сумму дочерних планов "Резервный фонд", "Отпуск", "Новая машина").

2. Каскадное Финансирование (Опционально/Продвинутая логика):

- **DistributionRule** может быть настроено на пополнение родительского плана. Средства, поступившие на родительский план, могут затем автоматически распределяться между дочерними планами согласно их **priority** или дополнительным суб-правилам.
- *Для начала, проще реализовать прямое пополнение конкретных планов (будь то родительский или дочерний). Каскадное финансирование – это усложнение, которое можно добавить позже.*

3. Правила Иерархии:

- План не может быть своим собственным родителем или предком (предотвращение циклов).
- При архивировании родительского плана, его дочерние планы могут либо также архивироваться, либо становиться планами верхнего уровня (на усмотрение пользователя или по системному правилу).

Обеспечение Простоты Использования:

- **Создание Плана:** Пользователь вводит `name` и выбирает `type`. Поля `target_amount/target_date` появляются только для релевантных типов. `priority` может иметь значение по умолчанию.
- **Управление Денежными Потоками:**
 - **Прогнозирование:** Через `ForecastEntry` (для общих ожидаемых доходов/расходов) и `DistributionRule` (для направления этих виртуальных потоков в планы). Эти элементы настраиваются один раз для регулярных событий.
 - **Фактические Операции:** При добавлении `Transaction`:
 - Система может предлагать планы для связи (через `TransactionPlanLink`) на основе описания транзакции, суммы, или ближайших "ожидаемых" `ForecastEntry`.
 - Пользователь подтверждает или корректирует связь.
 - Для планов типа `budget_category` ("карманные расходы"): одна крупная транзакция (например, снятие наличных) может быть привязана к плану. Последующие мелкие расходы из этих наличных могут не требовать индивидуальной линковки в системе, если пользователь этого не хочет; их эффект уже учтен либо первоначальной привязкой, либо общим `ForecastEntry` расхода против этого плана.
- **Балансы:** Пользователь видит `ProjectedBalance` на графиках. `CurrentActualAllocatedAmount` помогает понять, сколько реальных денег где находится *сейчас*.

Возможные Проблемы и Их Решения/Соображения:

1. **Сложность расчета `ProjectedBalance` в реальном времени:** Для множества планов, сценариев и длительных периодов это может быть медленно.
 - **Решение:** Периодически (например, ежедневно или при изменениях в `ForecastEntry/DistributionRule`) предварительно рассчитывать и сохранять снимки балансов (`PlanBalanceSnapshot`: `plan_id`, `date`, `scenario_tag`, `balance`). Это значительно ускорит отображение графиков.
2. **Обработка прогнозируемых овердрафтов:** Если `ProjectedBalance` плана уходит в минус.
 - **Решение:** Система должна это четко показывать. Это сигнал пользователю для корректировки планов, доходов, расходов или правил распределения. Система не "запрещает" прогнозируемый минус, а информирует о нем.
3. **Согласованность `ProjectedBalance('today')` и `CurrentActualAllocatedAmount`:**
 - В идеальном мире, если все фактические транзакции до сегодняшнего дня корректно связаны с планами, эти два значения должны быть очень близки для базового сценария. `ProjectedBalance('today', 'baseline')` учитывает все прогнозы до сегодня. `CurrentActualAllocatedAmount` учитывает все фактические привязки.
 - Расхождения между ними указывают на то, что либо не все фактические операции были учтены в привязках к планам, либо какие-то прогнозные события не произошли/произошли иначе. Это ценная информация для пользователя для анализа и корректировки.

Эта модель сущности `Plan` стремится быть достаточно простой в своих полях, но мощной за счет взаимодействия с `ForecastEntry`, `DistributionRule` и механизмами расчета/отображения балансов.

Ключевым является разделение между виртуальным/прогнозным слоем и слоем фактических операций, а также четкое определение поведения на основе **Plan.type**.